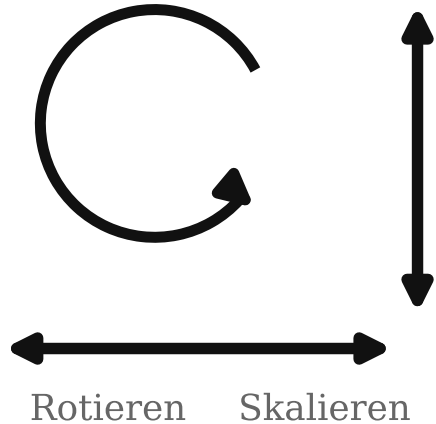


Singulärwertzerlegung

Singular Value Decomposition (SVD)

Erklärt anhand von Bildkomprimierung

Rotation und Skalierung



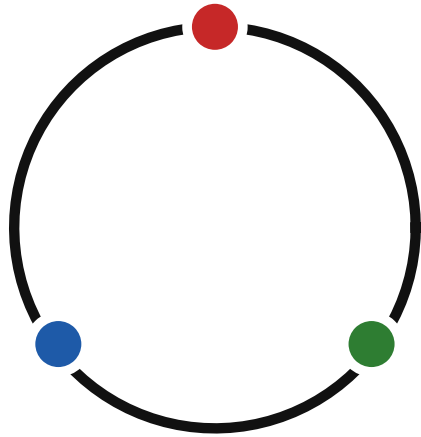
Eine **Rotation** ist eine Drehung um einen Winkel.
Eine **Skalierung** streckt oder staucht entlang einer Achse.

Diese beiden Operationen sind die geometrischen Bausteine, mit denen wir gleich eine lineare Abbildung in einfache Schritte zerlegen.

Die Matrixschreibweise kommt später; hier geht es zuerst nur um die sichtbare Wirkung.

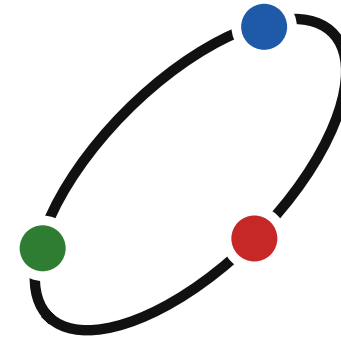
Von einer Form zur anderen

Bevor wir Bilder komprimieren, betrachten wir ein Rätsel: Wie kommt man nur durch Rotationen und Skalierungen entlang der Achsen von der linken Grafik zur rechten?



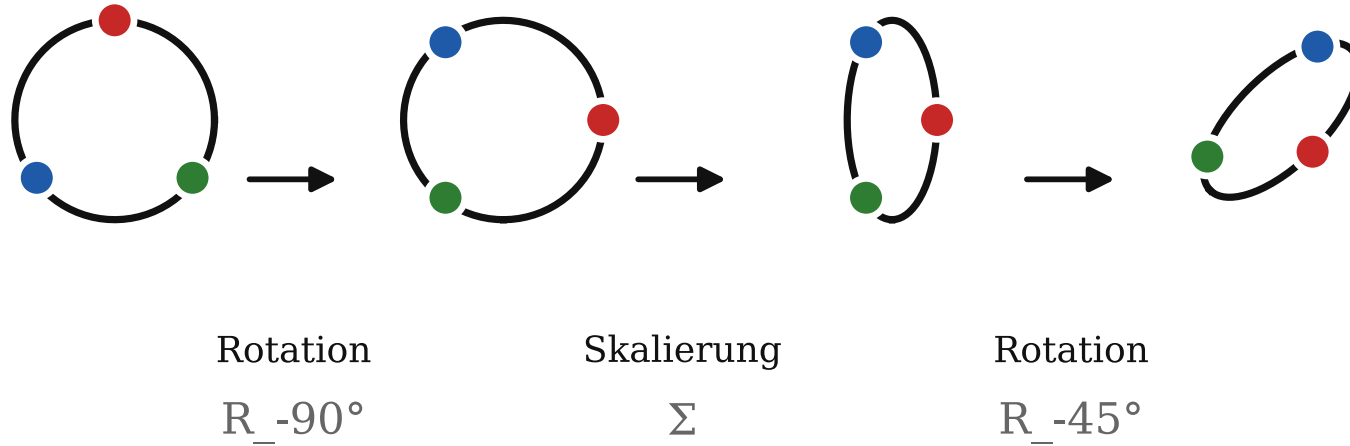
Ausgangsform

lineare Transformation



Zielbild

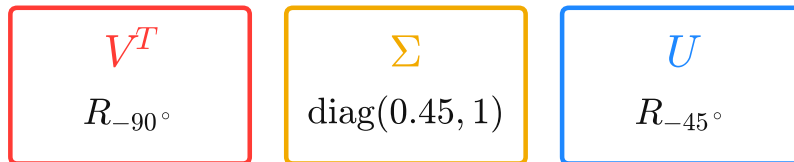
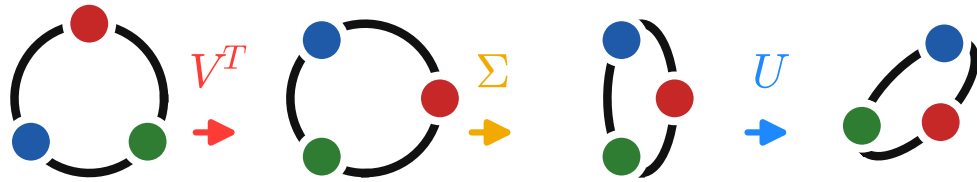
Rotation, Skalierung, Rotation



$$R_{-90^\circ} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \quad \Sigma = \begin{pmatrix} 0.45 & 0 \\ 0 & 1 \end{pmatrix} \quad R_{-45^\circ} = \begin{pmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{pmatrix}$$

Doch was hat das mit SVD zu tun?

Die Idee der SVD



Im Kern ist das das Prinzip der SVD:

$$A = U \Sigma V^T$$

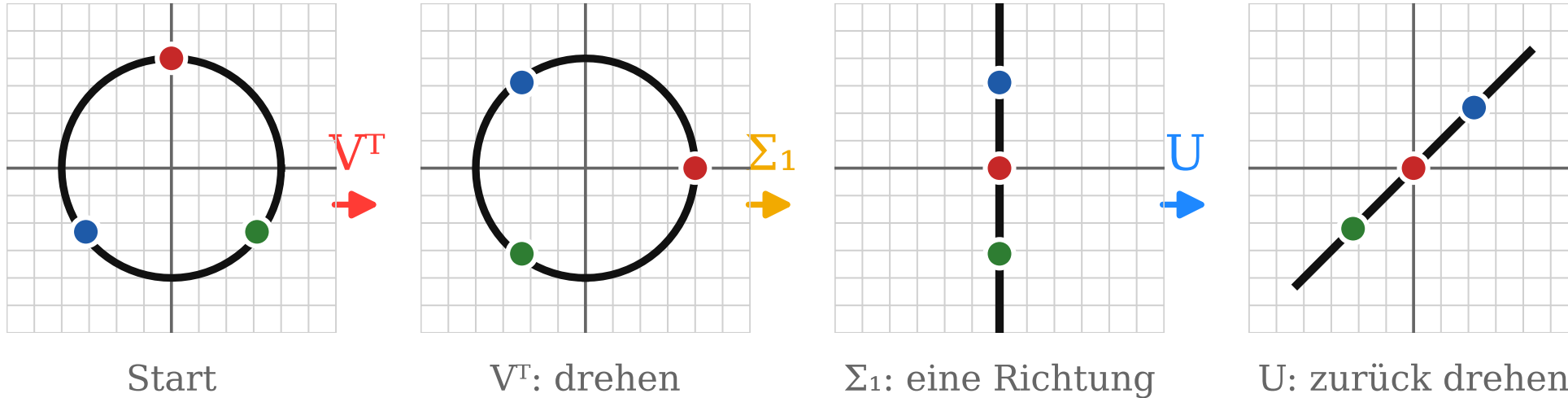
Eine lineare Abbildung wird in drei lesbare Bausteine zerlegt: erst eine Rotation oder Spiegelung, dann eine Skalierung, dann erneut eine Rotation oder Spiegelung.

Unser Ablauf von der vorherigen Folie ist dabei ein **didaktisches Beispiel im Stil der SVD**. Er zeigt das Grundprinzip, ist aber nicht die kanonische SVD, weil die Singulärwerte dort nach Größe sortiert werden und die stärkste Skalierung zuerst steht.

Für dieses Beispiel entsteht die Gesamtmatrix durch Multiplikation:

$$A = R_{-45^\circ} \begin{pmatrix} 0.45 & 0 \\ 0 & 1 \end{pmatrix} R_{-90^\circ} \approx \begin{pmatrix} -0.707 & 0.318 \\ -0.707 & -0.318 \end{pmatrix}$$

Dimensionsreduktion



Warum ist die Zerlegung nützlich, wenn eine einzige Matrix A die Abbildung auch direkt beschreibt?

Der entscheidende Punkt liegt in Σ : Wenn ein Skalierungswert auf 0 gesetzt wird, bleibt eine Richtung erhalten und die andere verschwindet.

Aus einer Fläche wird eine Linie. Genau diese Idee steckt hinter Dimensionsreduktion und später hinter Bildkomprimierung.

Rang-1-Matrix

1	2	3	4
-1	-2	-3	-4
2	4	6	8
10	20	30	40

16 Zahlen

=

1
-1
2
10

·

1	2	3	4
---	---	---	---

8 Zahlen

$$A = uv^T$$

Alle Zeilen sind Vielfache von (1, 2, 3, 4).

Deshalb hat die Matrix Rang 1: 4 Dimensionen, aber nur eine unabhängige Richtung.

Diese Matrix wirkt zuerst wie 16 einzelne Zahlen. Tatsächlich steckt aber viel weniger unabhängige Information darin.

Die Matrix hat also vier Zeilen und vier Spalten, aber nur **eine unabhängige Richtung**. Deshalb ist ihr Rang gleich 1.

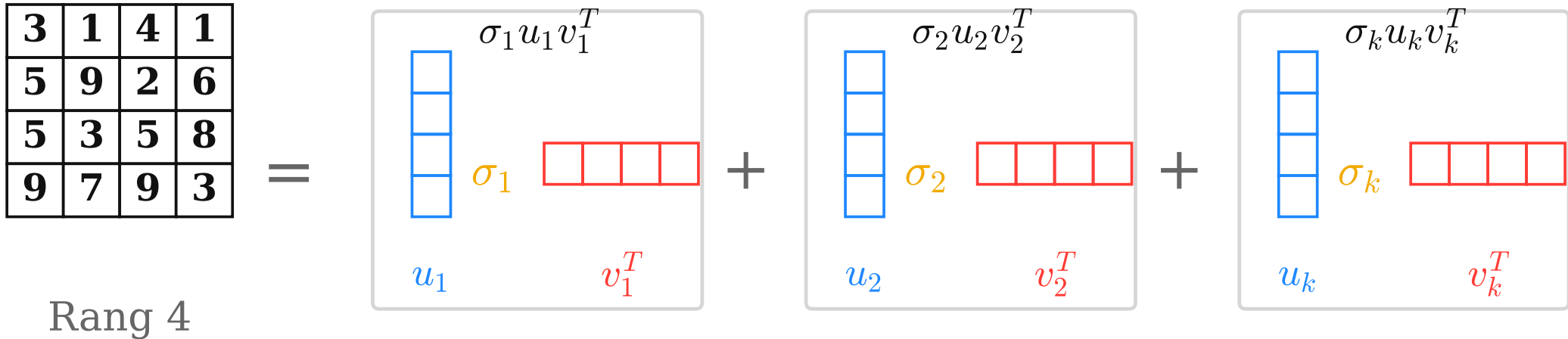
Alle Zeilen sind Vielfache der ersten Zeile:

$$(1, 2, 3, 4), \quad -(1, 2, 3, 4), \quad 2(1, 2, 3, 4), \quad 10(1, 2, 3, 4).$$

Statt 16 Zahlen speichern wir nur zwei Vektoren mit insgesamt 8 Zahlen:

$$A = uv^T.$$

Höherer Rang: Summe aus Rang-1-Matrizen



$$A_k = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_k u_k v_k^T$$

Bei einer Matrix mit Rang 4 sind die Zeilen nicht mehr alle Vielfache voneinander. Die einfache Zerlegung aus der vorherigen Folie reicht dann nicht mehr aus.

Das Grundprinzip bleibt aber gleich: Wir beschreiben die Matrix als Summe mehrerer Rang-1-Matrizen.

$$A_k = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_k u_k v_k^T$$

Für $k = r$ ist das die vollständige Zerlegung. Für $k < r$ entsteht eine Näherung: Je mehr Bausteine wir addieren, desto genauer wird sie. Für Kompression speichern wir nur die wichtigsten Bausteine und lassen kleine Beiträge weg.

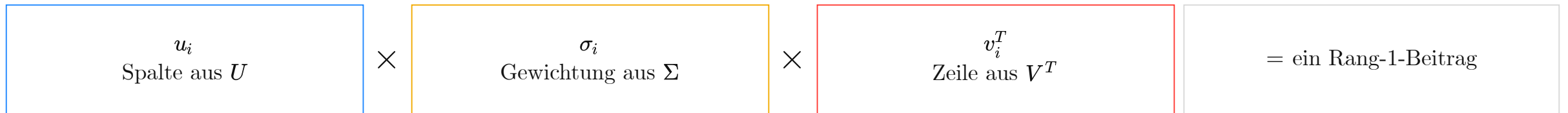
SVD als Summe von Rang-1-Beiträgen

Die Produktform

$$A = U \Sigma V^T$$

ist gleichbedeutend mit einer Summe aus einzelnen Rang-1-Matrizen:

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T.$$



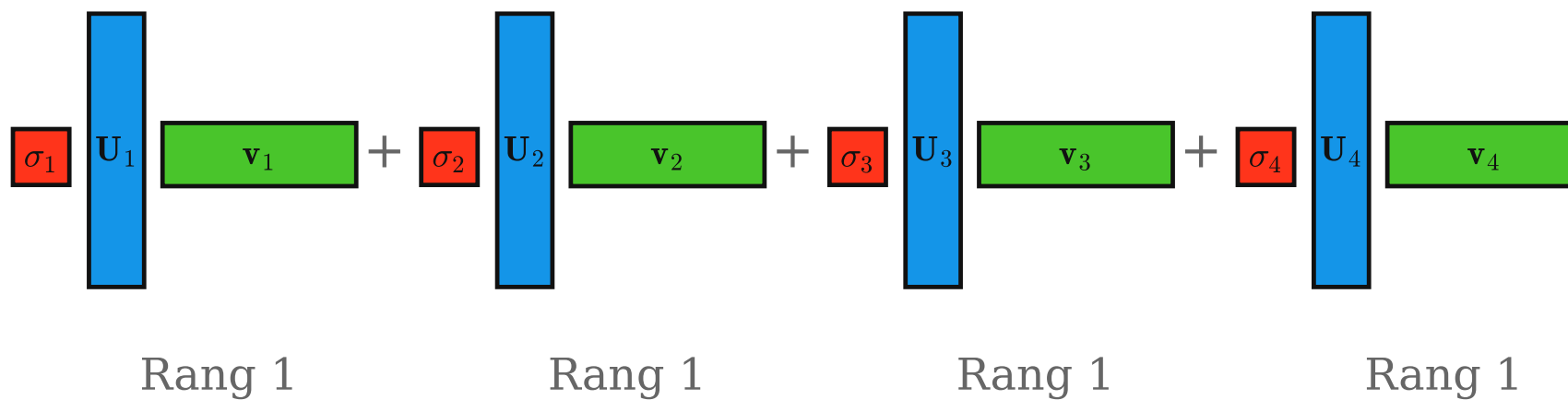
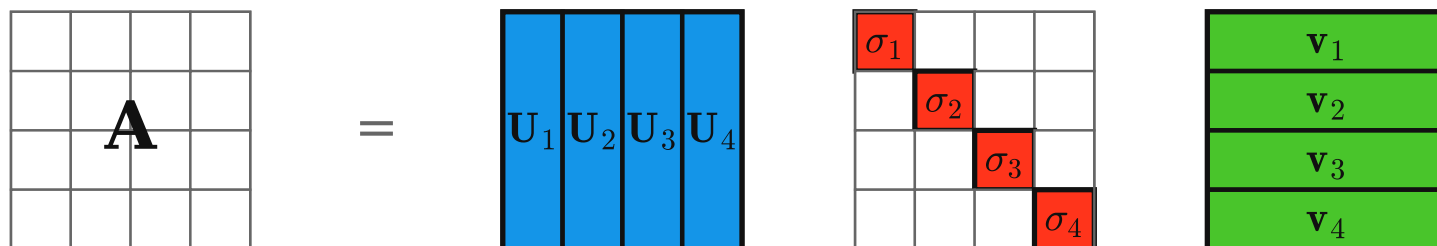
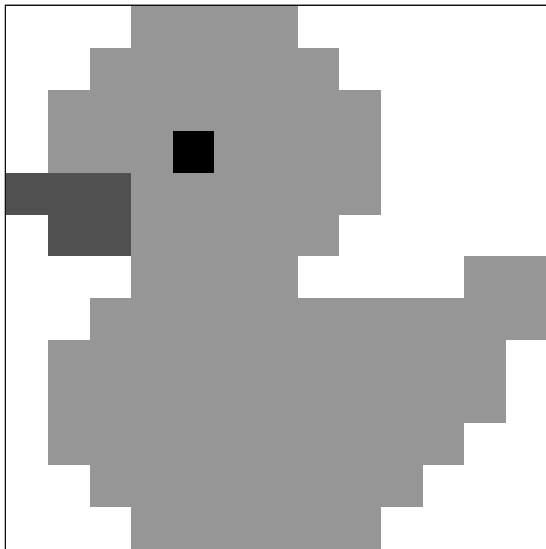


Bild als Matrix



Bild



255	255	255	150	150	150	150	255	255	255	255	255	255	255
255	255	150	150	150	150	150	150	255	255	255	255	255	255
255	150	150	150	150	150	150	150	150	255	255	255	255	255
255	150	150	150	0	150	150	150	150	255	255	255	255	255
80	80	80	150	150	150	150	150	150	255	255	255	255	255
255	80	80	150	150	150	150	150	255	255	255	255	255	255
255	255	255	150	150	150	150	255	255	255	255	150	150	
255	255	150	150	150	150	150	150	150	150	150	150	150	
255	150	150	150	150	150	150	150	150	150	150	150	150	255
255	150	150	150	150	150	150	150	150	150	150	150	150	255
255	150	150	150	150	150	150	150	150	150	150	150	255	255
255	255	150	150	150	150	150	150	150	150	255	255	255	
255	255	255	150	150	150	150	150	150	255	255	255	255	

Matrixwerte 0 bis 255

Bis hierhin haben wir Matrizen als Summen von Rang-1-Beiträgen betrachtet. Jetzt wenden wir genau diese Idee auf ein Bild an.

Ein Graustufenbild ist nichts anderes als eine Matrix A : Jeder Eintrag ist ein Pixelwert zwischen 0 und 255.

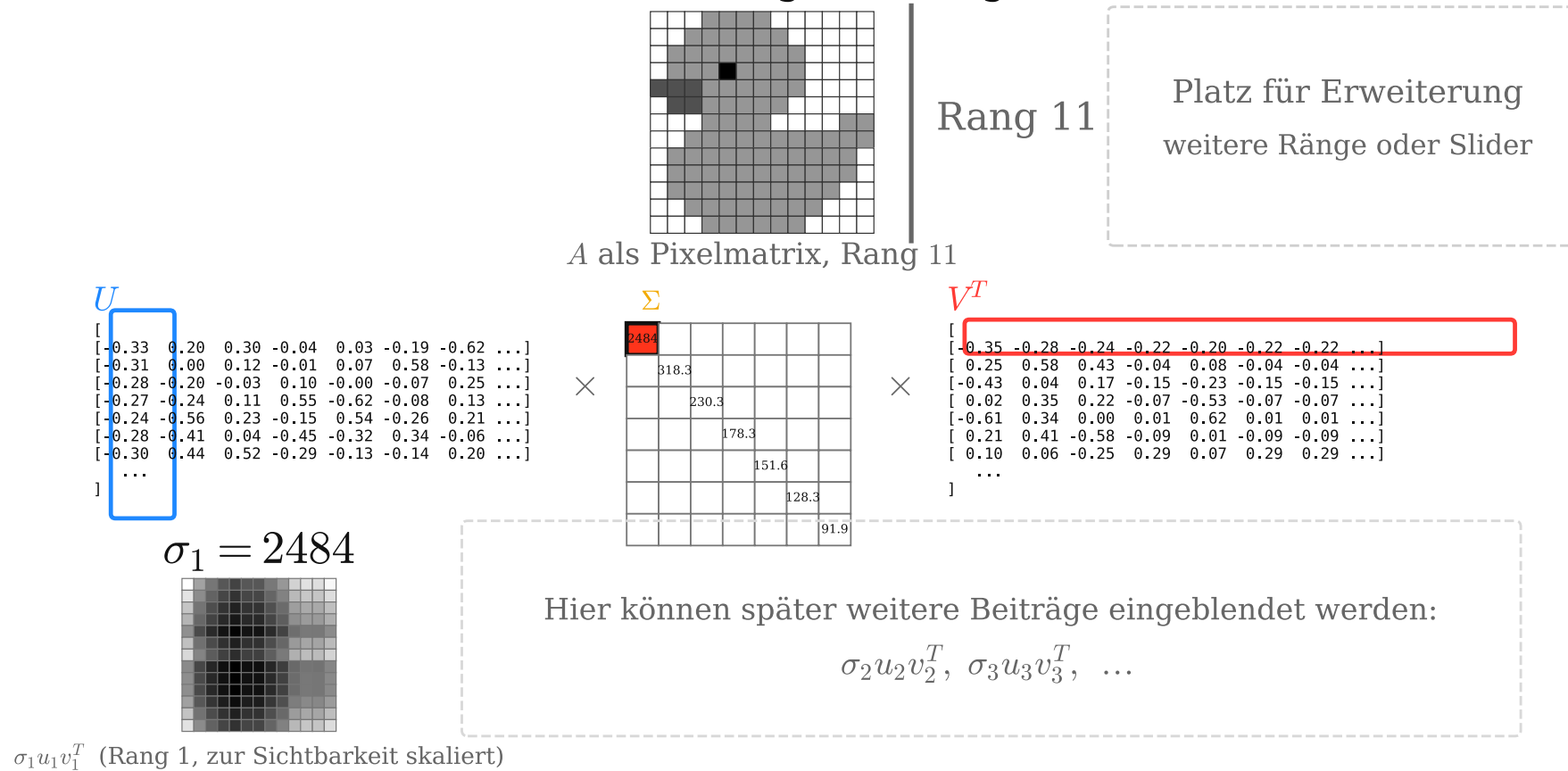
$$A = U\Sigma V^T,$$

zerlegt diese Pixelmatrix in geordnete Bildbausteine.

Die größten Singulärwerte beschreiben die wichtigsten Strukturen der Ente. Für die Kompression speichern wir nur diese stärksten Beiträge und lassen kleinere Details weg.

Auf der nächsten Folie sieht man diese Beiträge einzeln.

SVD der Entenmatrix: erster Rang-1-Beitrag



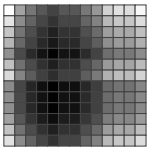
Rang-1-Beiträge der Ente: Konzept A

Einzelne Beiträge der SVD

Beitrag 1

Rang 1

$$\sigma_1 = 2484$$

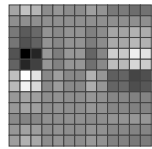


$$\sigma_1 u_1 v_1^T$$

Beitrag 2

Rang 1

$$\sigma_2 = 318$$

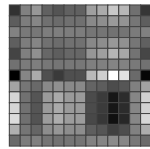


$$\sigma_2 u_2 v_2^T$$

Beitrag 3

Rang 1

$$\sigma_3 = 230$$



$$\sigma_3 u_3 v_3^T$$

Wichtig:

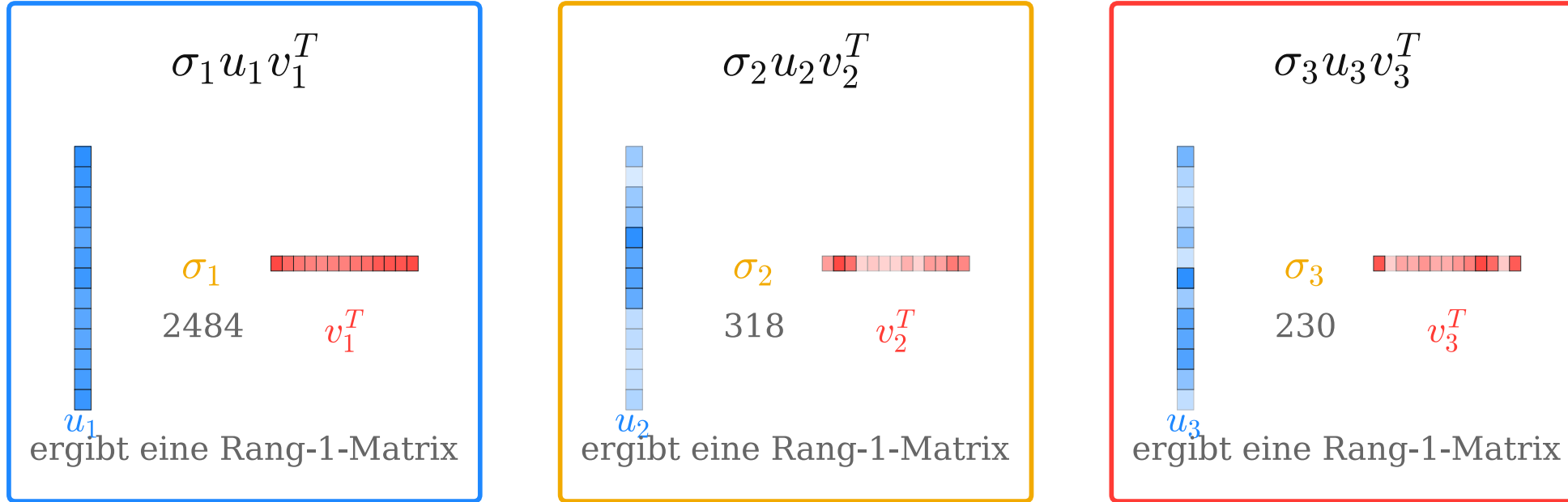
Jeder einzelne Beitrag ist eine Matrix vom Rang 1.

Die Beiträge unterscheiden sich nicht im Rang, sondern in ihrem Singulärwert und in den beiden Richtungsvektoren.

Die kleinen Enten links sind einzeln normalisiert, damit man die Muster sehen kann. Sie sind noch nicht aufsummiert.

Rang-1-Beiträge der Ente: Konzept B

Ein Rang-1-Beitrag besteht immer aus drei Teilen



Jeder Term

$$\sigma_i u_i v_i^T$$

ist eine Matrix vom Rang 1.

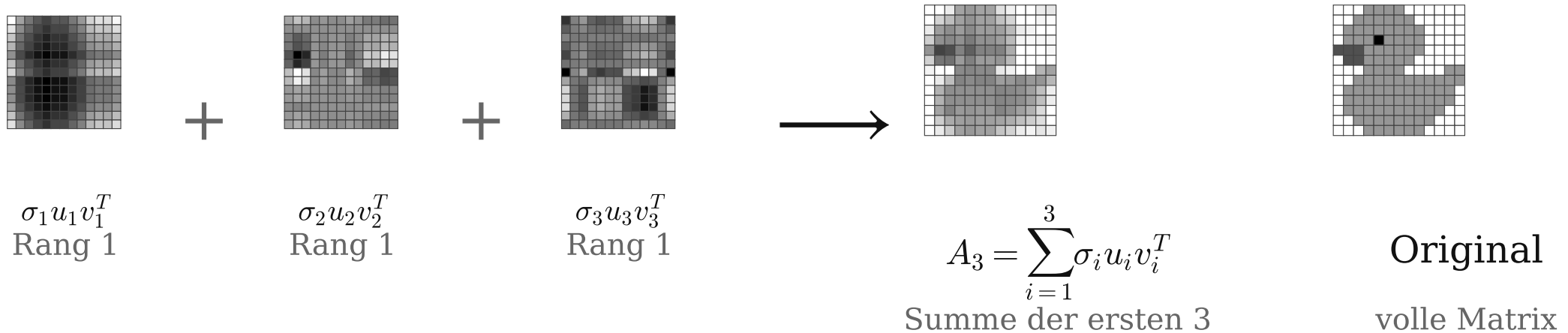
Der Singulärwert σ_i gewichtet den Beitrag. Die Vektoren u_i und v_i^T bestimmen das konkrete Muster.

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$$

Die Bildmatrix entsteht durch Addition dieser Rang-1-Beiträge.

Rang-1-Beiträge der Ente: Konzept C

Einzeln sind es Rang-1-Muster; durch Addition entsteht die Rekonstruktion



Links sind die ersten drei Beiträge einzeln dargestellt. Jeder einzelne Beitrag hat Rang 1.

Erst durch Addition entsteht eine bessere Näherung:

$$A_3 = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \sigma_3 u_3 v_3^T.$$

Diese Variante macht den Unterschied zwischen **einzelnen Beitrag** und **aufsummierter Rekonstruktion** sichtbar.

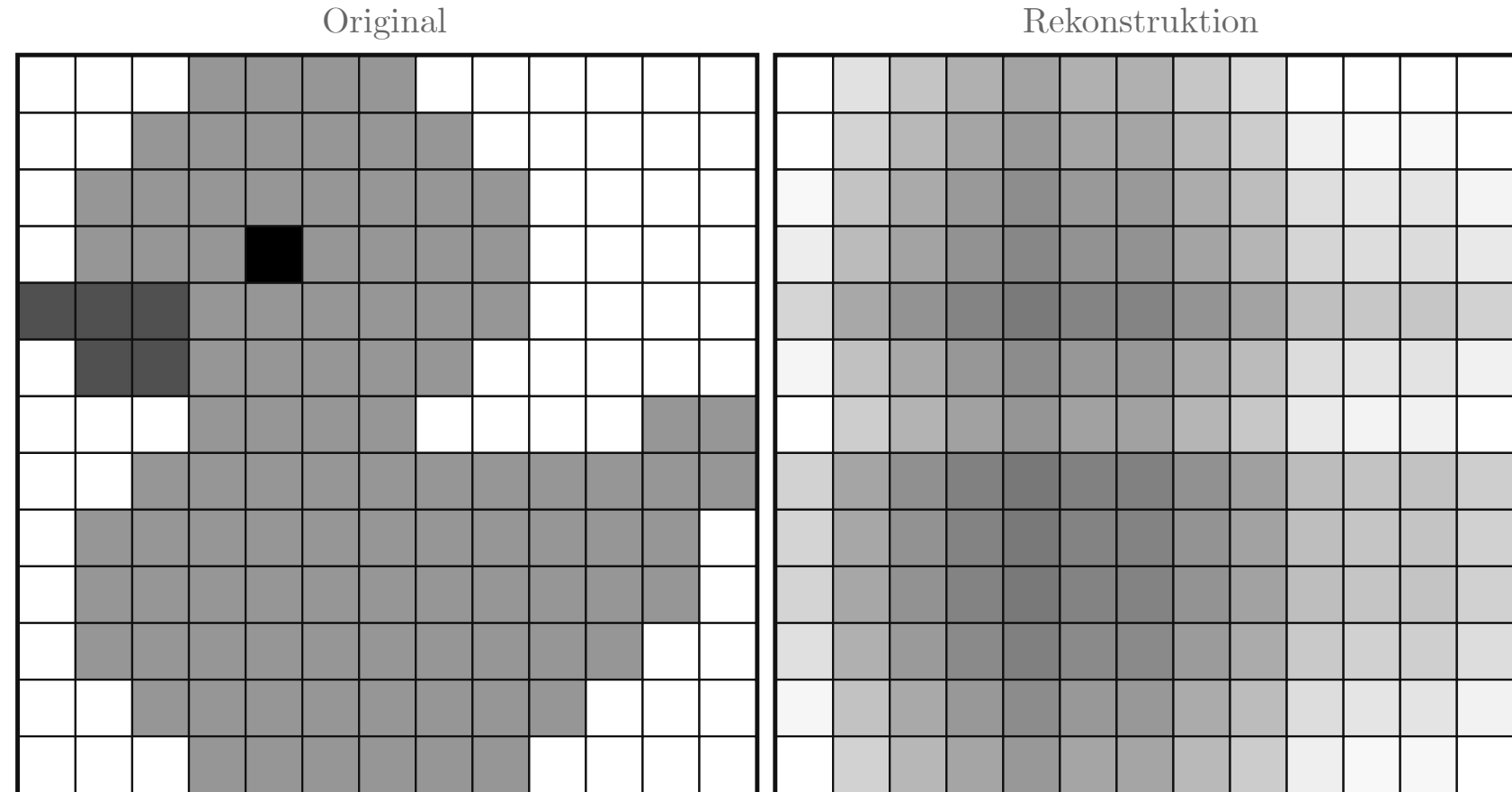
Rang-k-Näherung der Ente

Bei einem Bild ist A die Matrix der Pixelwerte.

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T.$$

Mit jedem Rang kommt ein weiteres Muster dazu. Kleine Ränge speichern wenig, verlieren aber Details; größere Ränge nähern sich der Originalmatrix an.

Rang $k = 1$  27 statt 169 Zahlen



Rang-k-Näherung von Albert Einstein

Bei einem hochauflösten Bild wird die Pixelmatrix deutlich größer. Ein Originalbild mit m Zeilen und n Spalten speichert ungefähr $m \cdot n$ Helligkeitswerte.

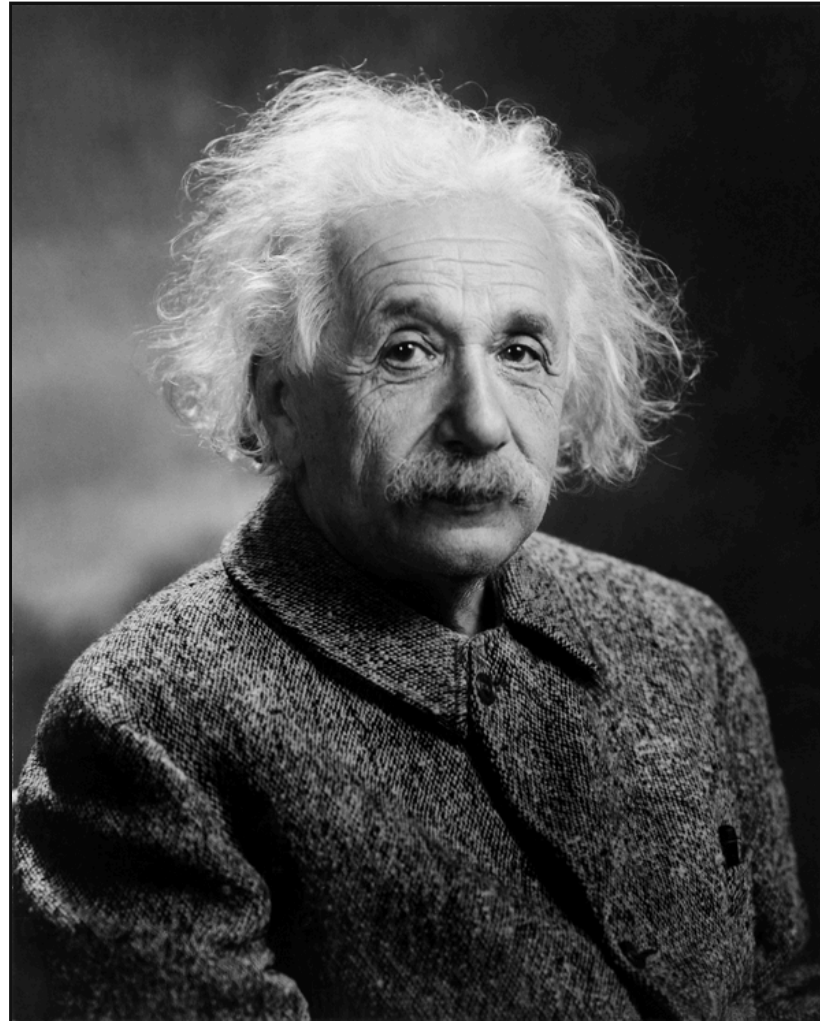
Original: $m \cdot n$ Rang- k : $k(m + n + 1)$

Bei hoher Auflösung lohnt sich diese Speicherung stärker: Für kleine k behalten wir nur wenige wichtige Bildmuster, sparen aber sehr viele Pixelwerte ein.

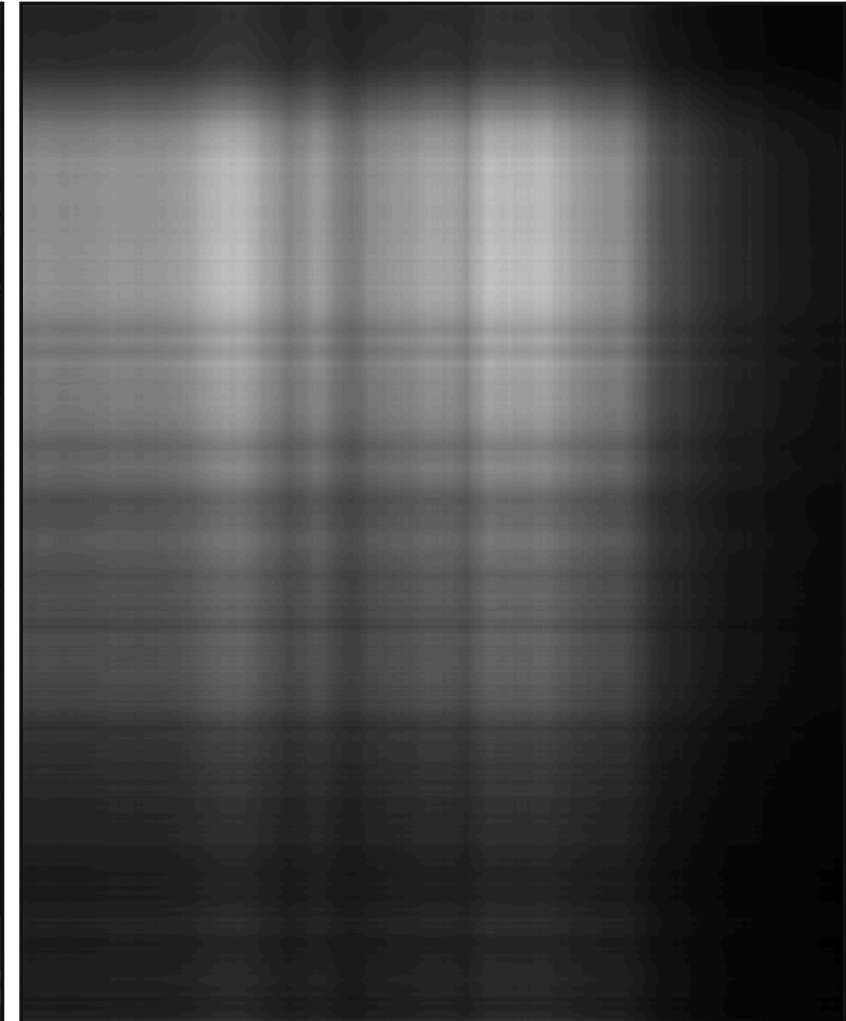
Je höher wir k wählen, desto mehr Details, Kanten und feine Kontraste kommen zurück. Gleichzeitig steigt aber auch die Datenmenge wieder.

Rang $k = 1$  1344 statt 445800 Zahlen

Original



Rekonstruktion



Von der Matrix zur SVD

Bis hierhin haben wir die SVD benutzt. Jetzt klären wir, wie man aus einer gegebenen Matrix A die drei Teile findet:

$$A = UV^T.$$

Die äquivalente Rang-Form ist:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T.$$

Für ein Bild ist A die Pixelmatrix:

$$A = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix}.$$

Die SVD sucht darin geordnete Muster: erst die wichtigsten, dann immer feinere Details.

Wie berechnet man U , Σ und V^T aus A ?

Ziel: drei einfache Bausteine

Für $A \in \mathbb{R}^{m \times n}$ suchen wir

$$A = U \Sigma V^T,$$

mit

$$U^T U = I, \quad V^T V = I, \quad \Sigma = \begin{pmatrix} \sigma_1 & 0 & \dots \\ 0 & \sigma_2 & \dots \\ \vdots & \vdots & \ddots \end{pmatrix}.$$

U

orthogonale Richtungen im Zielraum

Σ

Stärken der Richtungen

V^T

orthogonale Richtungen im Eingaberaum

Orthogonale Matrizen sind wie Rotationen oder Spiegelungen: Sie verändern Längen nicht. Die eigentliche Streckung steckt vollständig in den Singularwerten.

Schritt 1: Warum $A^T A$?

Wir suchen Eingaberichtungen v_i , die durch A besonders stark oder schwach gestreckt werden.

Die Länge nach der Abbildung ist

$$\|Av\|^2.$$

Das lässt sich umformen:

$$\|Av\|^2 = (Av)^T (Av) = v^T A^T Av.$$

Deshalb betrachten wir

$$A^T A.$$

Diese Matrix ist symmetrisch:

$$(A^T A)^T = A^T A.$$

Außerdem ist sie positiv semidefinit:

$$v^T A^T Av = \|Av\|^2 \geq 0.$$

Schritt 2: Eigenzerlegung

Für symmetrische Matrizen gilt der **Spektralsatz**:

Sie besitzen orthogonale Eigenvektoren und lassen sich diagonalisieren.

$$A^T A = V \Lambda V^T.$$

Die Spalten von V sind Eigenvektoren:

$$A^T A v_i = \lambda_i v_i.$$

Diese Eigenvektoren sind die Eingaberichtungen der SVD.

Warum ist das nützlich?

$$\|Av_i\|^2 = v_i^T A^T A v_i = \lambda_i.$$

Die Eigenwerte λ_i sagen also, wie stark A die Richtung v_i streckt.

Schritt 3: Singulärwerte

Aus den Eigenwerten von $A^T A$ werden die Singulärwerte:

$$\sigma_i = \sqrt{\lambda_i}.$$

Da $A^T A$ positiv semidefinit ist, gilt

$$\lambda_i \geq 0.$$

Also sind die Singulärwerte reell und nichtnegativ.

Sortiert wird absteigend:

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0.$$

Dadurch stehen die wichtigsten Muster zuerst:

$$\Sigma = \begin{pmatrix} \sigma_1 & 0 & \dots \\ 0 & \sigma_2 & \dots \\ \vdots & \vdots & \ddots \end{pmatrix}.$$

Schritt 4: Zielrichtungen u_i

Wenn v_i bekannt ist, zeigt Av_i in den Zielraum.

Die Länge kennen wir schon:

$$\|Av_i\| = \sigma_i.$$

Also normieren wir:

$$u_i = \frac{1}{\sigma_i} Av_i.$$

Damit gilt

$$Av_i = \sigma_i u_i.$$

Alle Richtungen zusammen:

$$AV = U\Sigma.$$

Da V orthogonal ist, gilt $V^{-1} = V^T$. Also:

$$A = U\Sigma V^T.$$

Algorithmus für eine Matrix

1. Berechne $A^T A$.
2. Löse $A^T A v_i = \lambda_i v_i$.
3. Setze $\sigma_i = \sqrt{\lambda_i}$ und sortiere absteigend.
4. Baue $V = (v_1, \dots, v_n)$ und Σ .
5. Berechne $u_i = \frac{1}{\sigma_i} A v_i$ für $\sigma_i > 0$.

Für Bilder macht man das numerisch. Die Idee bleibt:

Richtungen finden $\rightarrow$ Stärken sortieren $\rightarrow$ erste k behalten.

So entsteht die Kompression:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T.$$

Numerischer Bezug: QR

Die Herleitung nutzt die Eigenzerlegung von $A^T A$:

$$A^T A = V \Lambda V^T.$$

In der Numerik berechnet man Eigenwerte oft iterativ. Ein wichtiger Baustein ist die **QR-Zerlegung**:

$$B = QR,$$

wobei Q orthogonal und R eine obere Dreiecksmatrix ist.

Wichtig:

QR ist **nicht** die SVD.

Aber QR ist verwandt über orthogonale Matrizen. QR-Verfahren können Eigenwerte stabil approximieren, und daraus kann man wiederum Singulärwerte gewinnen.

Für große Bilder verwendet man meist direkte oder iterative SVD-Algorithmen, statt $A^T A$ naiv per Hand auszurechnen.

Warum Rang- k optimal ist

Die SVD liefert nicht irgendeine Kürzung, sondern die beste Rang- k -Approximation.

Der Satz dahinter heißt **Eckart-Young-Mirsky**:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$$

minimiert den Fehler unter allen Matrizen mit Rang höchstens k :

$$\|A - B\|_F \quad \text{für} \quad \text{rang}(B) \leq k.$$

Der Fehler hängt direkt an den weggelassenen Singulärwerten:

$$\|A - A_k\|_F^2 = \sigma_{k+1}^2 + \sigma_{k+2}^2 + \dots$$

Deshalb funktioniert Bildkompression gut, wenn viele spätere Singulärwerte klein sind: Wir verlieren wenig sichtbare Information, sparen aber viele Daten.

Mini-Beispiel: reine Skalierung

Nehmen wir

$$A = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}.$$

Dann ist

$$A^T A = \begin{pmatrix} 9 & 0 \\ 0 & 1 \end{pmatrix}.$$

Die Eigenwerte sind $\lambda_1 = 9$ und $\lambda_2 = 1$.

Also sind die Singulärwerte

$$\sigma_1 = 3, \quad \sigma_2 = 1.$$

Die Eigenvektoren sind

$$v_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad v_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Damit ist

$$U = I, \quad \Sigma = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}, \quad V^T = I.$$

Mini-Beispiel mit Rotation

Wenn wir vor und nach der Skalierung drehen, wirkt die Matrix komplizierter:

$$A = R_{-45^\circ} \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix} R_{-90^\circ}.$$

Die SVD erkennt trotzdem:

Rotation $\rightarrow$ Skalierung $\rightarrow$ Rotation.

Die Singulärwerte bleiben

$$\sigma_1 = 3, \quad \sigma_2 = 1.$$

Was sich ändert, sind die Richtungen:

$$V^T = R_{-90^\circ}, \quad U = R_{-45^\circ}.$$

Das verbindet die geometrischen Folien am Anfang mit der Bildkompression am Ende.